

3

STORAGE CLASS & FUNCTION

3.1 Memory Organization of C Program

3.1.1 Code Segment

- It is also known as text segment.
- It contains executable instructions and this segment is a read only segment.
- Usually, this section is sharable.

3.1.2 Uninitialized data segment

- This section contains all static and global variables that are not initialized by the programmer and hence initialized to zero (default value).

3.1.3 Initialized data segment

- This section contains all static and global variables that are initialized by the programmer.

3.1.4 Stack

- In this local variables are stored and some other information is also saved.
- A set of values stored for a function is called as stack frame which atleast contain return address.

3.1.5 Heap

- This area is responsible for dynamic memory allocation. The heap area is managed by malloc(), calloc(), realloc() and free() which may use brk() and sbrk() system calls.

3.1.6 Static Memory

- Compile time allocation
- Static variable
- Global Variable

3.2 Storage Classes

Storage Class	Scope	Life Time	Default	Storage area
auto	Local	Within function	Garbage Value	RAM
static	Local	Till end of main program	0	RAM

extern	Global	Till end of main program	0	RAM
Register	Local	Within function	Garbage	Register

- Stack Overflow: Abnormal Termination
- If conflict between global and local variable occurs, then local variable gets preference.
- Declaration of a register variable is only a recommendation/request not a command.
- Cannot apply '&' operator with register variable.
- No physical memory is allocated using 'extern' keyword.
- Extern declaration is mandatory when an external variable is referred before it is defined or if it is defined in some other source file from the file where it is being used.
- Declaration of an external variable tells about the properties like its type, while definition leads to storage allocation.

3.3 Recursion

Definition: When the body of a function call the function itself directly or indirectly.

- Certain arguments for which the function does not call itself are called as base argument, base values.
- In general, for recursion to be non-cyclic whenever a function calls itself the formal arguments must get closer to the base argument.

Example 1:

Consider the factorial code:

```
int factorial (int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial (n - 1);
}
```

If we call factorial (3), it will call factorial of 2 and so on ...the argument will get closer to 0 i.e., the base argument.

- Recursion code is shorter than iterative code.
- Overhead is present.
- Some standard problems are best suited to be solved by using recursion for example- Tower of Hanoi, Factorial, merge Sort.

3.4 Static and dynamic scoping

3.4.1 Static Scoping

Static scoping is also called as lexical scoping. In this scoping, the binding of a variable can be determined by the program text. In this type of scoping compiler first looks in the current block (local), then in global variables i.e., local and then ancestors' strategy is followed.

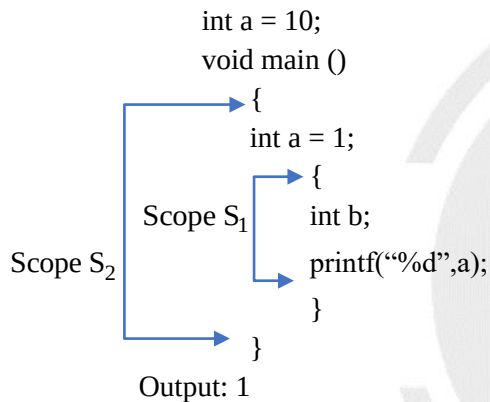
Example 1:

```
int a = 10;
void main ()
{
    int a = 1;
    {
        int a = 2
        printf(“%d”,a);
    }
}
```

Output: 2

As printf is referring to a, compiler first check in the current block & gets a variable whose value is 2.

Example 2:



Compiler looks for ‘a’ in scope S₁, because S₁ does not have variable a, it will look into higher scope S₂ that contains a variable name a whose value is 1.

3.4.2 Dynamic Scoping

In this type of scoping, the compiler first searches the current block and then successively searches all calling functions.

Example:

```
int i;
program main ()
{
    i = 10;
    call f ();
}
procedure f ()
{
    int c = 20;
    call g();
}
Procedure g() {
    print i;
}
```

Assuming that the above program is written in a hypothetical programming language which allows global variables



and dynamic scoping, let's try to find the output.
The order of function calls is:

main () $\xrightarrow{\text{calling}}$ f() $\xrightarrow{\text{calling}}$ g()

g() is printing i, which is not present inside current block. So, because of dynamic scoping compiler will go to the function that calls g() i.e., compiler will search f() for variable i. Hence 20 is printed.

□□□

